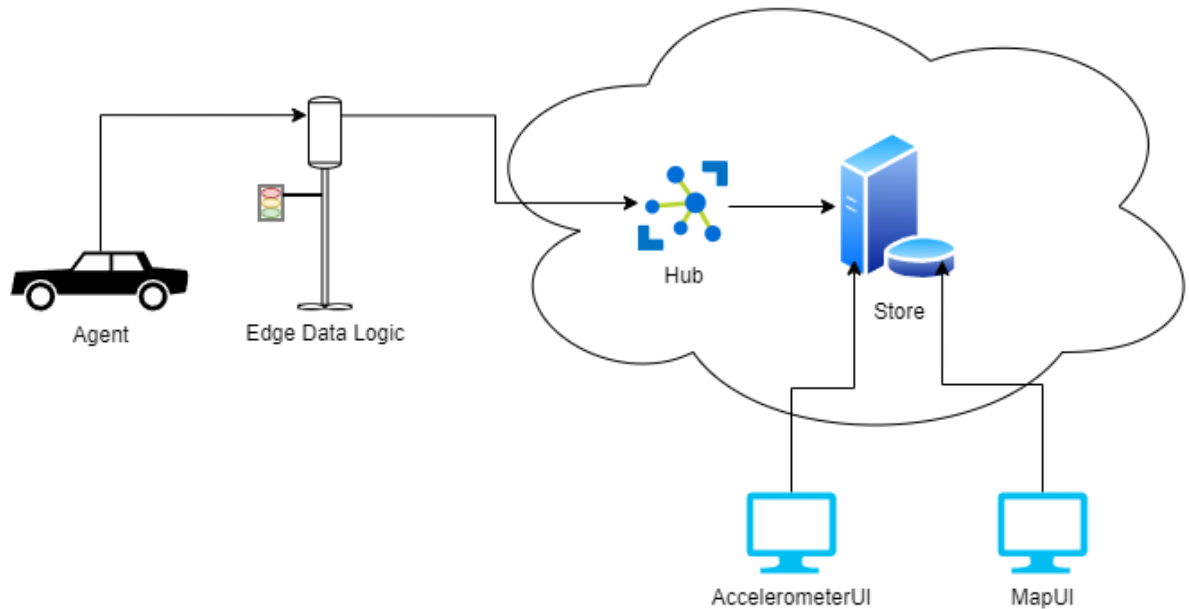


Лабораторна робота №3

Опис проекту



Завдання – розробити комплексну систему моніторингу та аналізу стану дорожнього покриття. Система може мати можливість збирати дані про різні параметри, такі як нерівності, тріщини, вибоїни та інші дефекти, які можуть вплинути на якість дороги. Зібрані дані слід проаналізувати, щоб визначити закономірності та тенденції, які можуть допомогти у прогнозуванні потреб у технічному обслуговуванні дорожньої мережі. Система також може мати можливість генерувати звіти та сповіщення для відповідних органів влади та/або водіїв.

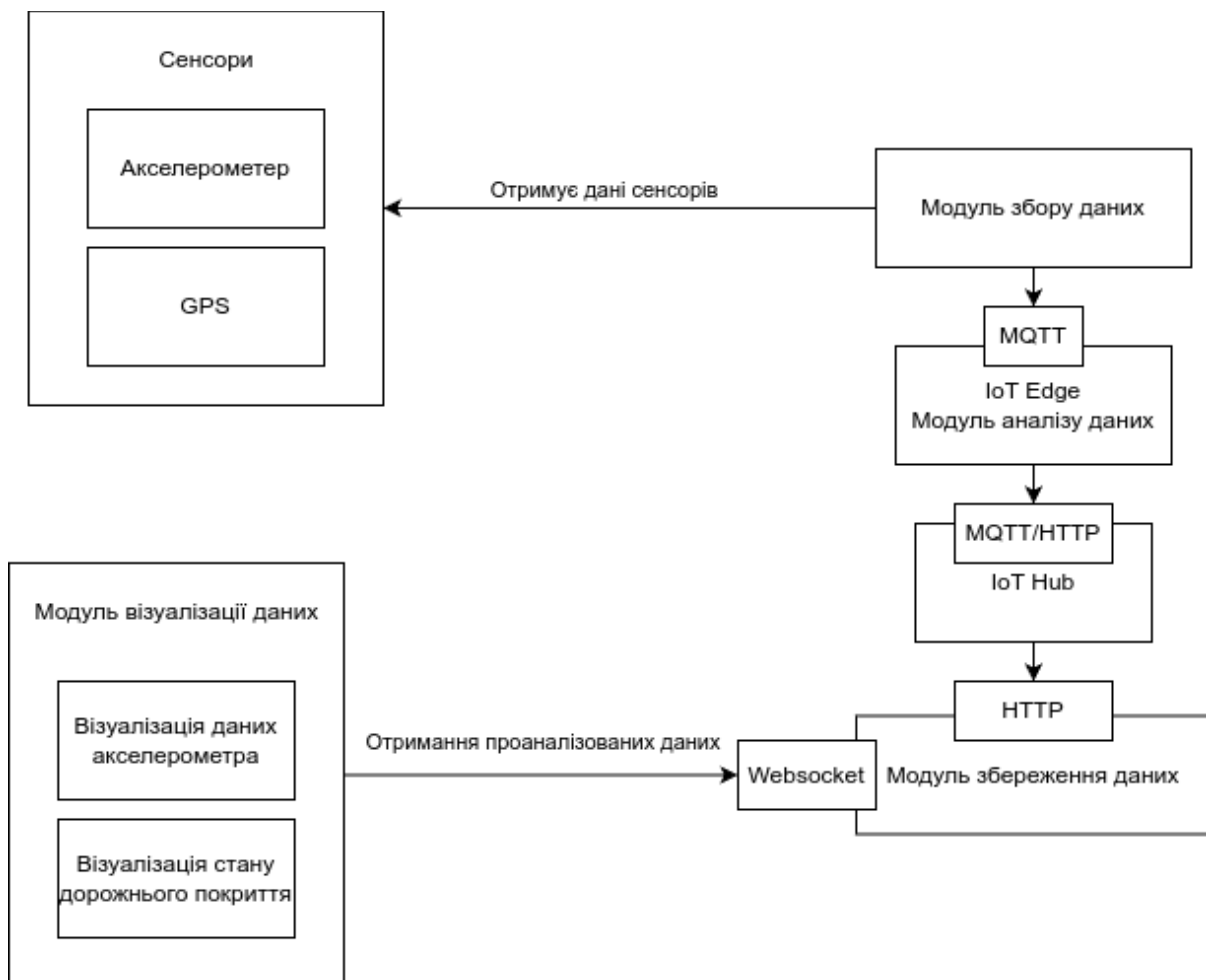
Метою цієї системи є підвищення безпеки дорожнього руху, зниження витрат на технічне обслуговування та покращення загального досвіду водіння для автомобілістів.

В систему входять пристрій на машині (stm32), edge пристой, хмарні або on-prem сервіси.

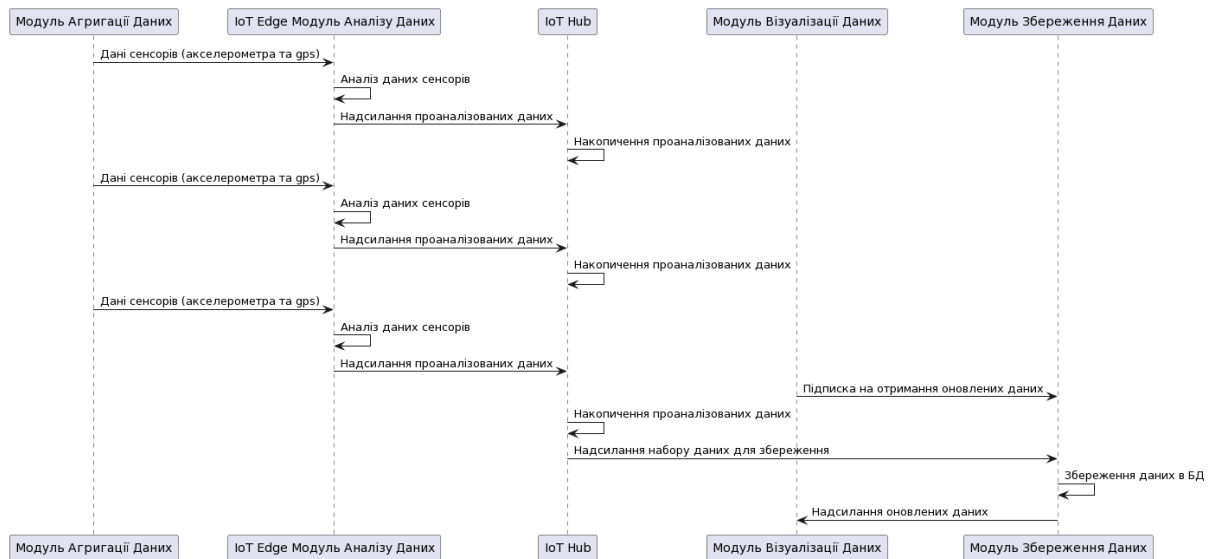
Система містить наступні частини (дозволяється розширювати):

- Модуль збору даних (Agent) - програма агент на машині, яка знімає показники сенсорів (наприклад раз в 100мс) та відправляє їх на сервер.

- Модуль первинної обробки даних (Edge Data Logic) - програма яка займається первинною обробкою даних, в нашому випадку визначає стан дорожнього покриття та відправляє проаналізовані дані на Hub.
- Модуль накопичення даних (Hub) - сервіс який займається накопиченням та аналізом (preprocessing) отриманих даних від Agent та подальшим зберіганням оброблених даних в БД. Перед збереженням дани потрібно накопичити (наприклад 10 записів), так як такі дані простіше аналізувати і запис в БД множини даних буде швидшим.
- Store - арі для отримання та збереження даних в БД.
- Модуль візуалізації (Accelerometer UI, Map UI) - інтерфейс візуалізації (моніторингу) результатів системи.



Архітектура системи



Юзкейс роботи

Постановка задачі

Пітрібно реалізувати частину Hub. Сервіс який займається накопиченням отриманих даних від Agent/EdgeDataLogic та подальшим зберіганням оброблених даних в БД. Перед збереженням діні потрібно накопичити (наприклад 10 записів), так як такі дані простіше аналізувати і запис в БД множини даних буде швидшим.

Кроки виконання

Надалі буде показана реалізація на python, проте ніхто не забороняє використовувати той технологічний стек який для вас є оптимальним.

До початку роботи потрібно встановити [Docker](#) та [MQTT Explorer](#).

Hub може отримувати дані з MQTT або через Http, або по іншому протоколу. Буде представлено два варіанти для MQTT та Http протоколів, проте студент можу вибрати і реалізувати комунікацію через інший протокол.

1. Створення віртуального середовища. Пакет **virtualenv** повинен бути встановлений в системі.

```
python -m venv venv
```

2. Активация середовища

```
source ./venv/bin/activate # linux  
.\venv\Scripts\activate.bat # windows
```

3. Встановимо необхідні залежності(бібліотеки)

```
pip install fastapi  
pip install "uvicorn[standard]"  
pip install pydantic  
pip install paho-mqtt
```

4. Збережемо залежності в requirements.txt

```
pip freeze > requirements.txt
```

5. Структура проекту буде наступною

```
main.py  
config.py  
app  
    adapters  
        store_api_adapter.py  
    entities  
        agent_data.py  
        processed_agent_data.py  
    interfaces  
        store_api_gateway.py  
docker  
    docker-compose.yaml  
Dockerfile
```

requirements.txt
README.md

6. Реалізуємо моделі даних (беремо з попередніх лабораторних).

agent_data.py

```
1  from datetime import datetime
2  from pydantic import BaseModel, field_validator
3
4
5  class AccelerometerData(BaseModel):
6      x: float
7      y: float
8      z: float
9
10
11  class GpsData(BaseModel):
12      latitude: float
13      longitude: float
14
15
16  class AgentData(BaseModel):
17      accelerometer: AccelerometerData
18      gps: GpsData
19      timestamp: datetime
20
21      @classmethod
22      @field_validator('timestamp', mode='before')
23      def parse_timestamp(cls, value):
24          # Convert the timestamp to a datetime object
25          if isinstance(value, datetime):
26              return value
27          try:
28              return datetime.fromisoformat(value)
29          except (TypeError, ValueError):
30              raise ValueError(
31                  "Invalid timestamp format. Expected ISO 8601 format
32                  (YYYY-MM-DDTHH:MM:SSZ).")
33
```

processed_agent_data.py

```
1  from pydantic import BaseModel
2  from app.entities.agent_data import AgentData
3
4
5  class ProcessedAgentData(BaseModel):
6      road_state: str
7      agent_data: AgentData
```

7. Наступним кроком визначимо інтерфейс для роботи з Store.

store_gateway.py

```
1  from abc import ABC, abstractmethod
2  from typing import List
3  from app.entities.processed_agent_data import ProcessedAgentData
4
5
6  class StoreGateway(ABC):
7      """
8      Abstract class representing the Store Gateway interface.
9      All store gateway adapters must implement these methods.
10     """
11
12     @abstractmethod
13     def save_data(self, processed_agent_data_batch:
14     List[ProcessedAgentData]) -> bool:
15         """
16         Method to save the processed agent data in the database.
17         Parameters:
18             processed_agent_data_batch (ProcessedAgentData): The processed
19         agent data to be saved.
20         Returns:
21             bool: True if the data is successfully saved, False otherwise.
22         """
23         pass
```

8. Реалізуємо StoreApiAdapter. Необхідно імплементувати запит до StoreApi для збереження даних. А саме зробити post запит на

‘{store_host}/processed_agent_data’ з списком елементів ProcessedAgentData.

store_api_adapter.py

```
1  import json
2  import logging
3  from typing import List
4
5  import pydantic_core
6  import requests
7
8  from app.entities.processed_agent_data import ProcessedAgentData
9  from app.interfaces.store_gateway import StoreGateway
10
11
12  class StoreApiAdapter(StoreGateway):
13      def __init__(self, api_base_url):
14          self.api_base_url = api_base_url
15
16      def save_data(self, processed_agent_data_batch: List[ProcessedAgentData]):
17          # Make a POST request to the Store API endpoint with the processed data
```

9. Визначимо конфігурації.

config.py

```

1  import os
2
3
4  def try_parse_int(value: str):
5      try:
6          return int(value)
7      except Exception:
8          return None
9
10
11  # Configuration for the Store API
12  STORE_API_HOST = os.environ.get("STORE_API_HOST") or "localhost"
13  STORE_API_PORT = try_parse_int(os.environ.get("STORE_API_PORT")) or
14  8000
15  STORE_API_BASE_URL = f"http://{STORE_API_HOST}:{STORE_API_PORT}"
16
17  # Configure for Redis
18  REDIS_HOST = os.environ.get("REDIS_HOST") or "localhost"
19  REDIS_PORT = try_parse_int(os.environ.get("REDIS_PORT")) or 6379
20
21  # Configure for hub logic
22  BATCH_SIZE = try_parse_int(os.environ.get("BATCH_SIZE")) or 20
23
24  # MQTT
25  MQTT_BROKER_HOST = os.environ.get("MQTT_BROKER_HOST") or "localhost"
26  MQTT_BROKER_PORT =
    try_parse_int(os.environ.get("MQTT_BROKER_PORT")) or 1883
    MQTT_TOPIC = os.environ.get("MQTT_TOPIC") or
    "processed_agent_data_topic"

```

10. Реалізація main.py


```

1  import logging
2  from typing import List
3
4  from fastapi import FastAPI
5  from redis import Redis
6  import paho.mqtt.client as mqtt
7
8  from app.adapters.store_api_adapter import StoreApiAdapter
9  from app.entities.processed_agent_data import ProcessedAgentData
10 from config import STORE_API_BASE_URL, REDIS_HOST, REDIS_PORT,
11 BATCH_SIZE, MQTT_TOPIC, MQTT_BROKER_HOST, \
12     MQTT_BROKER_PORT
13
14 # Configure logging settings
15 logging.basicConfig(
16     level=logging.INFO, # Set the log level to INFO (you can use logging.DEBUG
17 for more detailed logs)
18     format="[%(asctime)s] [% (levelname)s] [% (module)s] %(message)s",
19     handlers=[
20         logging.StreamHandler(), # Output log messages to the console
21         logging.FileHandler("app.log"), # Save log messages to a file
22     ],
23 )
24 # Create an instance of the Redis using the configuration
25 redis_client = Redis(host=REDIS_HOST, port=REDIS_PORT)
26 # Create an instance of the StoreApiAdapter using the configuration
27 store_adapter = StoreApiAdapter(api_base_url=STORE_API_BASE_URL)
28 # Create an instance of the AgentMQTTAdapter using the configuration
29
30 # FastAPI
31 app = FastAPI()
32
33
34 @app.post("/processed_agent_data/")
35 async def save_processed_agent_data(processed_agent_data:
36 ProcessedAgentData):
37     print(processed_agent_data)
38     redis_client.lpush(
39         "processed_agent_data",
40         processed_agent_data.model_dump_json()
41     )
42     processed_agent_data_batch: List[ProcessedAgentData] = []
43     if redis_client.llen("processed_agent_data") >= BATCH_SIZE:
44         for _ in range(BATCH_SIZE):
45             processed_agent_data = ProcessedAgentData.model_validate_json(
46                 redis_client.lpop("processed_agent_data")
47             )
48             processed_agent_data_batch.append(processed_agent_data)
49     store_adapter.save_data(
50         processed_agent_data_batch=processed_agent_data_batch
51     )

```

```

52     return {"status": "ok"}
53
54
55 # MQTT
56 client = mqtt.Client()
57
58
59 def on_connect(client, userdata, flags, rc):
60     if rc == 0:
61         logging.info("Connected to MQTT broker")
62         client.subscribe(MQTT_TOPIC)
63     else:
64         logging.info(f"Failed to connect to MQTT broker with code: {rc}")
65
66
67 def on_message(client, userdata, msg):
68     batch_data = None
69     try:
70         payload: str = msg.payload.decode("utf-8")
71         # Create ProcessedAgentData instance with the received data
72         processed_agent_data = ProcessedAgentData.model_validate_json(payload,
73 strict=True)
74
75         redis_client.lpush(
76             "processed_agent_data",
77             processed_agent_data.model_dump_json()
78         )
79         processed_agent_data_batch: List[ProcessedAgentData] = []
80         if redis_client.llen("processed_agent_data") >= BATCH_SIZE:
81             for _ in range(BATCH_SIZE):
82                 processed_agent_data = ProcessedAgentData.model_validate_json(
83                     redis_client.lpop("processed_agent_data")
84                 )
85                 processed_agent_data_batch.append(processed_agent_data)
86             store_adapter.save_data(
87                 processed_agent_data_batch=processed_agent_data_batch
88             )
89             return {"status": "ok"}
90         except Exception as e:
91             logging.info(f"Error processing MQTT message: {e}")
92
93
94 # Connect
95 client.on_connect = on_connect
96 client.on_message = on_message
97 client.connect(MQTT_BROKER_HOST, MQTT_BROKER_PORT)
98
99
100 # Start
101 client.loop_start()

```

11.Добавимо Dockerfile для запуску в docker.

Dockerfile

```
1  # Use the official Python image as the base image
2  FROM python:latest
3  # Set the working directory inside the container
4  WORKDIR /app
5  # Copy the requirements.txt file and install dependencies
6  COPY requirements.txt .
7  RUN pip install --no-cache-dir -r requirements.txt
8  # Copy the entire application into the container
9  COPY . .
10 # Run the main.py script inside the container when it starts
11 CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

12.Налаштуємо docker-compose.yaml для локального запуску та тестування програми.

docker-compose.yaml

```
1  version: "3.9"
2  name: "road_vision__hub"
3  services:
4    mqtt:
5      image: eclipse-mosquitto
6      container_name: mqtt
7      volumes:
8        - ./mosquitto:/mosquitto
9        - ./mosquitto/data:/mosquitto/data
10       - ./mosquitto/log:/mosquitto/log
11     ports:
12       - 1883:1883
13       - 9001:9001
14     networks:
15       mqtt_network:
16
17   postgres_db:
18     image: postgres:latest
19     container_name: postgres_db
20     restart: always
21     environment:
22       POSTGRES_USER: user
23       POSTGRES_PASSWORD: pass
24       POSTGRES_DB: test_db
25     volumes:
26       - postgres_data:/var/lib/postgresql/data
27       - ./db/structure.sql:/docker-entrypoint-initdb.d/structure.sql
28     ports:
29       - "5432:5432"
30     networks:
31       db_network:
32
33
34   pgadmin:
35     container_name: pgadmin4
36     image: dpage/pgadmin4
37     restart: always
38     environment:
39       PGADMIN_DEFAULT_EMAIL: admin@admin.com
40       PGADMIN_DEFAULT_PASSWORD: root
41     volumes:
42       - pgadmin-data:/var/lib/pgadmin
```

```
43     ports:
44         - "5050:80"
45     networks:
46         db_network:
47
48
49     store:
50         container_name: store
51         build: ../../store
52         depends_on:
53             - postgres_db
54         restart: always
55         environment:
56             POSTGRES_USER: user
57             POSTGRES_PASSWORD: pass
58             POSTGRES_DB: test_db
59             POSTGRES_HOST: postgres_db
60             POSTGRES_PORT: 5432
61     ports:
62         - "8000:8000"
63     networks:
64         db_network:
65         hub_store:
66
67     redis:
68         image: redis:latest
69         container_name: redis
70         ports:
71             - "6379:6379"
72         networks:
73             hub_redis:
74
75     hub:
76         container_name: hub
77         build: ../
78         depends_on:
79             - mqtt
80             - redis
81             - store
82         environment:
83             STORE_API_HOST: "store"
84             STORE_API_PORT: 8000
```

```
85     REDIS_HOST: "redis"
86     REDIS_PORT: 6379
87     MQTT_BROKER_HOST: "mqtt"
88     MQTT_BROKER_PORT: 1883
89     MQTT_TOPIC: "processed_data_topic"
90     BATCH_SIZE: 1
91     ports:
92     - "9000:8000"
93     networks:
94     mqtt_network:
95     hub_store:
96     hub_redis:
97
98     networks:
99     mqtt_network:
100    db_network:
101    hub_store:
102    hub_redis:
103
104    volumes:
105    postgres_data:
106    pgadmin-data:
```

13.Запуск програму в docker

`cd docker`

`docker-compose up --build`

14.Перевірка працездатності в MQTT Explorer або через Swagger та PgAdmin.

Очікувані результати в протоколі

1. Посилання на git репозиторій з проектом
2. Реалізація StoreGateway

3. Показ результатів роботи в MQTT Explorer, Swagger, PgAdmin
4. Висновки